UFUG2106 ·Round 1 Local Reproduction Report

Fixed-order ROBDD Baseline ·Mathematics, Implementation & Stage 1 Evidence

UFUG2106 ·Truth Table to Circuit ·Local Reproduction Archive

2026-11-11

目录

1. 研究问题与成本模型 1
2. Round 1：算法设计与数学依据 2
3. 实验环境与可追溯性 3
4. 官方测评数据 4
5. 验证覆盖、证据链与可信度边界 6
6. 局限性、后续研究动机与阶段性结论 6
7. 复现命令、证据索引与后续引用 7

文档性质：基于用户于 2026-11-11 上传的 WSL 本地复现 ZIP 中的原始日志、Evaluator CSV、Netlist、源码快照，以及与该版本对应的算法设计文档。本文是已核验的技术记录，不代表官方最终评分、全局最优性证明或隐藏测试表现。算法的数学正确性说明与实际运行的经验事实严格区分。

1. 研究问题与成本模型

完整真值表描述的是一个多输出 Boolean Function：

$$F : \{0,1\}^n \longrightarrow \{0,1\}^m, \qquad F(x) = (f_0(x), \dots, f_{m-1}(x)).$$

题目要求把它实现为有限、有向无环的门级 Netlist。只允许 **AND2、OR2、NOT** 三类门；编号 0 表示常量 0，编号 1 到 n 是输入变量 $x_0, \dots, x_{n-1}$，新门的输入只能引用更早的信号。一个输出可以直接指向输入或常量，不必多造一层门。真值表按 MSB-first 的变量顺序枚举所有 2^n 行。

对电路 C，使用当前测试案例的门成本 a_g, d_g：

$$A(C) = \sum_{g \in G(C)} a_g, \qquad T(v) = \max_{u \in \text{fanin}(v)} T(u) + d_v, \qquad D(C) = \max_{j=0}^{m-1} T(y_j).$$

输入和常量的到达时间为 0；单输入 NOT 门只取其唯一输入的到达时间。共享 Gate 只计算一次 Area。$D(C)$ 是 **Circuit Delay**，程序执行所需的 **Runtime** 是另一个独立指标。老师的 `status=PASS` 与 `target_met=yes` 也是两个不同字段：正确电路可以超过 Target Delay 而仍得到 PASS。

本研究的核心目标是在满足 $C(x) = F(x)$ 的前提下探索低 Area、低 Delay 与受控 Runtime；并未假定老师采用跨案例 Area 简单相加作为成绩。

2. Round 1：算法设计与数学依据

2.1 为什么能够从任意完整 Truth Table 构造电路?

对任意选定的输入变量 x_i，定义 Shannon Cofactors：

$$f_0 = f\big|_{x_i=0}, \qquad f_1 = f\big|_{x_i=1}.$$

Shannon 恒等式为：

$$\boxed{f = \neg x_i \wedge f_0 \ \vee \ x_i \wedge f_1.}$$

证明。当 $x_i = 0$ 时右式等于 f_0；当 $x_i = 1$ 时右式等于 f_1。布尔变量只有这两种取值，故恒等式对所有输入成立。递归分解余下变量，最终必到常量 0 或 1。这给出构造算法的数学基础。

例如，$f(x_0, x_1) = x_0 \oplus x_1$ 可写成：

$$f = (\neg x_0 \wedge x_1) \vee (x_0 \wedge \neg x_1),$$

所以没有原生 XOR Gate 也不影响表达能力。

2.2 固定顺序的 Reduced Ordered BDD

Round 1 固定变量顺序 $x_0 \prec x_1 \prec \cdots \prec x_{n-1}$。对每个输出，底向上组合真值表中的相邻 Cofactors，并通过两条规则实现 ROBDD 约简：

- **消除冗余测试：**若 `lo == hi`，则 $\mathrm{ITE}(x_i, lo, lo) = lo$，直接复用该子节点。
- **合并相同子函数结构：**以 `(variable, lo, hi)` 为 Key 的 Unique Table 合并相同三元组。

底向上构造的**归纳不变式**是：当前数组中的每个项，均表示相应已固定输入前缀下的 Cofactor。每次配对由 Shannon 恒等式组合这些项，最终根节点表示完整输出。多个输出共用同一个 BDD Unique Table，因而可能共享子函数节点。

这里的“规范性”须以固定变量顺序和完整约简为前提；不同变量排序下的 ROBDD 规模可以不同。

2.3 Gate Mapping 与两级共享

对每个 BDD 决策节点，递归生成

$$\mathrm{ITE}(x_i, lo, hi) = \neg x_i \wedge lo \vee x_i \wedge hi.$$

代码针对常量 Cofactor 使用精确的化简，例如：

$$\mathrm{ITE}(x, 0, h) = x \wedge h, \qquad \mathrm{ITE}(x, l, 1) = x \vee l.$$

还使用 Gate-level Structural Hashing，对 AND2 / OR2 的输入 Signal ID 排序后，以 `(gate type, fanin 1, fanin 2)` 建 Key 并复用相同门；同一个 NOT 信号也尽量复用。常量 1 可通过 `NOT 0` 获得。

需要区别：**BDD Unique Table 共享的是 Boolean Cofactor；Gate Hash 共享的是具体的门操作**。两种共享发生于不同抽象层，不能互相替代。

2.4 Netlist 清理、正确性和复杂度

构造结束后，从每个输出 Root 反向标记所有可达门，删除不可达门，最后按原有拓扑顺序重编号。由于删除的节点不能影响任何输出，功能不变；重编号只替换 Signal ID，不改变逻辑。

正确性证明框架：

1. 原始真值表数组的行索引与输入赋值一一对应。
2. 对剩余变量数作归纳，Cofactor 终端正确；每个 BDD 节点由 Shannon 恒等式恢复正确函数。
3. 同子节点消除与 Unique Table 合并均不改变函数；AND/OR/NOT Gate 映射及常量化简均为 Boolean 恒等式。
4. 结构哈希只共享计算，死门删除与重编号保持输出。因此整个 Netlist 与每个输入赋值的原始真值表严格一致。

复杂度边界：输入本身含 $m2^n$ 个输出位，因此读取至少需要 $\Omega(m2^n)$ 时间。本版本固定变量顺序的底向上配对数为每输出 $2^n - 1$，在哈希操作期望摊还 $O(1)$ 的模型下，BDD 构建期望 $O(m2^n)$ 时间；其他门处理还需额外成本。BDD 最坏规模依旧可能是指数级，而且上述分析**不保证**任意资源限制下程序能够完成。

3. 实验环境与可追溯性

日期与证据包： `2026-11-11T10:40:07+08:00`；`20261111_104006_Round1_local_results.zip`。

项目	本地记录
操作系统	Ubuntu 24.04.4 LTS / WSL2；Linux `6.6.87.2-microsoft-standard-WSL2`，x86_64
CPU	Intel Core Ultra 9 275HX；WSL 报告 24 个逻辑 CPU；综合过程为单线程

项目	本地记录
编译器	GCC 13.3.0；C++17；`g++ -O2 -std=c++17 -Wall -Wextra`
Python	3.12.3，执行独立验证及汇总
数据路径	Windows `E:` 盘对应的 WSL `/mnt/e/` 挂载目录
Evaluator	老师提供的 Linux Evaluator，复制到 `/tmp/ufug2106_evaluator` 执行
源码 SHA-256	`3989b677ba6ff7b533de4179c8b3fc1612878bc1b8fc8c1ae10e0`
Evaluator SHA-256	`142361d7ce18bc47dea6a07538db79a588b3afbf446526c1d1246`
原始 ZIP SHA-256	`0fdf760a7f920d1fb34e16e6b2dab64229ef432f681245572598f`

源码文件 `synth_source_snapshot.cpp` 与本轮冻结的 `src/synth.cpp` 经字节级 SHA-256 对照完全一致。报告复核 `official.csv` 中每个案例的 `status`、`area`、`delay`、`target_met`，并与同 ZIP 所含历史 `reference.csv` 比较。所有 25 行的 Area、Delay 与 Target Met 均保持一致；Runtime 不能要求逐字相等。

编译日志中的 `-Wmisleading-indentation` 警告指向一行多个 `if` 的排版，**不是编译失败**。这类警告仍值得在后续清理中修正，但本轮为了保持冻结版本不改动源文件。

4. 官方测评数据

4.1 总体结果

测评指标	本地测量	历史参考	解释
Official PASS	**25/25**	25/25	功能与格式通过老师的 Evaluator
Target Delay Met	**12/25**	12/25	`target_met` 单独统计，不等于 PASS
Area Sum	**1,103,455**	1,103,455	跨测试案例的描述性总和
Median Runtime	**5.455 ms**	7.263 ms	受主机与运行负载影响
Maximum Runtime	**95.293 ms**	94.723 ms	单案例运行时间最大值
独立检查 Area/Delay	**25/25 一致**	—	不依赖综合器内部成本累计
独立逻辑验证	**13 穷举 + 12 抽样**	—	大规模抽样不能当成穷举证明

4.2 逐案例结果

表中 `Slack = Target − Delay`；负值表示未达到 Target，并**不**意味着无效电路。`Runtime` 采用老师 Evaluator 给出的实际本地运行时间。

Case	n	m	Area	Delay	Target	Slack	Runtime (ms)	Target Met
001	15	14	135,885	71	77	+6	17.635	Yes
002	14	14	21,835	66	61	-5	8.918	No
003	14	14	28,206	66	64	-2	9.005	No
004	13	10	7,713	57	57	+0	5.042	Yes
005	8	5	1,594	36	36	+0	3.943	Yes
006	12	12	14,272	56	60	+4	5.428	Yes
007	20	15	231,712	89	89	+0	95.293	Yes
008	12	12	15,924	56	62	+6	4.797	Yes
009	12	6	33,212	56	59	+3	5.455	Yes
010	19	8	9,394	75	61	-14	28.753	No
011	10	5	620	47	40	-7	3.845	No
012	10	10	4,157	46	40	-6	4.371	No
013	14	14	11,826	67	70	+3	7.350	Yes
014	20	8	20,808	81	72	-9	58.155	No
015	16	16	38,556	76	81	+5	14.049	Yes
016	10	5	5,008	46	47	+1	3.698	Yes
017	13	12	573	26	25	-1	4.860	No
018	12	12	2,933	41	32	-9	5.402	No
019	18	8	23,692	76	73	-3	15.499	No
020	8	8	824	36	35	-1	4.071	No
021	15	14	69,666	72	72	+0	20.128	Yes
022	8	16	1,286	37	34	-3	3.540	No
023	8	4	262	35	24	-11	5.211	No
024	18	8	180,822	84	83	-1	50.744	No
025	15	15	242,675	71	76	+5	20.391	Yes

4.3 关键诊断

Target 未达标的 13 个案例： `002`, `003`, `010`, `011`, `012`, `014`, `017`, `018`, `019`, `020`, `022`, `023`, `024`。这些案例都获得 `PASS`，说明结果功能上有效，只是逻辑路径超过目标。

Case	Delay	Target	Delay Excess
002	66	61	+5
003	66	64	+2
010	75	61	+14
011	47	40	+7
012	46	40	+6
014	81	72	+9
017	26	25	+1
018	41	32	+9

Case	Delay	Target	Delay Excess
019	76	73	+3
020	36	35	+1
022	37	34	+3
023	35	24	+11
024	84	83	+1

其中 `case_010` 超过目标 **14** 个时间单位、`case_023` 超过 **11** 个时间单位；其余 11 个在表中列出。值得注意的是 `case_007` 的 Delay=89 恰好满足 Target=89，但它的 Area 为 231,712，同时也是本轮 Runtime 最大的案例（95.293 ms）。另两个面积较大的案例是 `case_025`（242,675）与 `case_024`（180,822）。

这些结果支持两种不同的瓶颈: 一类是关键路径过深, 另一类是正确电路仍可能面积过大。它们不构成特定 Target 不可达的数学证明。

5. 验证覆盖、证据链与可信度边界

1. **官方 Evaluator：** 25 个案例的 `status=PASS`，并提供 Area、Delay、Runtime、Target Met 等字段；判断是否满足 Target 必须读取 `target_met`，不能仅看 PASS。
2. **独立 Netlist Checker：**本地 ZIP 的 `independent_crosscheck.csv` 显示 25/25 个 Area/Delay 与官方逐行完全相等；`verification_summary.txt` 显示 **13 个案例穷举输入赋值，12 个案例固定种子抽样**，全部无问题。
3. **源码与证据版本：**本地 `synth_source_snapshot.cpp` 的哈希匹配冻结源码；原始 ZIP 保留了编译日志、Evaluator 日志、输出 Netlist 与输入案例的哈希清单。
4. **强度界限：** 12 个较大案例的第二套独立模拟是抽样，不可写成独立全表穷举；但老师的官方 Evaluator 在 25 个案例上均判定为 PASS。
5. **还没有证明的事：**Hidden Cases 的性能；全球最优 Area/Delay；任何输入的严格 Runtime/Memory 保证；官方课程成绩预测；任何尚未发生的个人或小组贡献。

6. 局限性、后续研究动机与阶段性结论

Round 1 固定 $x_0, \dots, x_{n-1}$ 的分解顺序，使用标准 Shannon 映射，**不包含** Round 2 的 Variable Influence、Two-level Expansion、Implicant SOP/POS 或 Portfolio，也不包含 Round 3 的 Complemented Edges、Affine Detection、Required-time Area Recovery。源码在 `AND` / `OR` 互补关系判断时可能为比较而生成暂时无用的 NOT 门，尽管之后可由 Reachability Pruning 删除，依旧有运行开销。

阶段性结论：Round 1 建立了可靠的任意真值表构造基线: 25/25 功能 PASS; 固定顺序与简单 Gate Mapping 并不足以稳定达到 Delay Target（仅 12/25）。这直接驱动 Round 2 研究“变量顺序 + 替代表达 + 多候选选择”，而不能把基线失败解读为电路无解。

7. 复现命令、证据索引与后续引用

```
BASE="/mnt/e/canfiles/Common courses study/大学通识/26-27 Fall/UFUG 2106 Discrete Math/assignment/
cd "$BASE/UFUG2106_Round1_Delivery"
bash run_round_local.sh /tmp/ufug2106_evaluator "$BASE/data/stage1"
```

建议长期保存: 20261111_104006_Round1_local_results.zip、其内 official.csv、independent_crosscheck.csv、verification_summary.txt、per_case_comparison.csv、synth_source_snapshot.cpp、environment.txt、case_sha256.txt、evaluator.log、build.log 和 25 份 outputs/。数学推导基于冻结源码与 UFUG2106_Round1_Delivery/docs/ALGORITHM_AND_MATH.md, 可与本报告共同组成 Final Report 的基线方法及实验章节。

参与声明: 当前证据只证明工具运行、源码版本和测评结果; 不能自动推断某位团队成员独立编写了源码。最终课程报告中须如实注明 AI 辅助、亲自开展的复现与实际团队分工。